Dukaan Pro - Database ERD (Version 2.1)

This version adds the **Online Store** (Release 1.2) to ERD v2. Everything is additive: **no existing table, column or rule from Release 1 changes**, so your finished Release 1 backend keeps working. PostgreSQL, one shared database, **shop_id on every table**.

> **Three releases, one database design**
>
> **Release 1 (19 tables, built):** shops, users, categories, brands, products, suppliers, purchases, purchase_items, supplier_payments, inventory_units, stock_levels, inventory_movements, customers, sales, sale_items, payments, expense_categories, expenses, cash_movements.
> **Release 1.1 (4 tables, dashed):** returns, return_items, refunds, cash_reconciliations.
> **Release 1.2 (2 new tables, pink dashed, plus 5 new product columns):** shop_store_settings, product_images.
> **Platform admin:** platform_admins is outside the shop tables (not drawn).

How to read the diagrams

Symbol	Meaning
Bar --- crow's foot	One-to-many. Bar side = parent (1), crow's foot side = child (many).
Dashed line	Optional link (the foreign key may be empty).
Dashed table border	Table is built in a later release: (R1.1) or (R1.2).
PK / FK / UQ	Primary key / Foreign key / Unique (UQ on a table means unique per shop unless the note says platform-wide).
Orange bold name	New in v2.1 (Release 1.2).
Pink table	Online Store tables.

Global design choices (unchanged)

- **IDs:** uuid.
- **Money:** bigint in paisa. The public API also returns paisa; the frontend divides by 100.
- **Time:** timestamptz in UTC.
- **No delete:** records are deactivated or cancelled. Store settings are switched off, not deleted.
- **Public data is an allowlist:** the public API selects named fields only. Nothing is ever returned by 'select all and remove'.

What changed from ERD v2 to v2.1

#	Change	Why
1	**New table shop_store_settings** (one row per shop): slug, is_enabled, display_name, about, logo, banner, WhatsApp, phone, address, city, map link, opening hours, Facebook, Instagram, show_exact_stock	Keeps every public field in one place, separate from the private shops table, so it is obvious what a customer can see.
2	**slug is unique**, lowercase, 3-40 characters, no reserved words	Each shop gets a unique store address (khalid-mobile, khalid-mobile-2). Reserved names (www, app, api, admin ...) cannot be taken.
3	**New table product_images**: url, sort_order	Storefront photos. Lowest sort_order is the cover.
4	**products.is_published** (default false)	A product is never public by accident. The owner chooses.
5	**products.published_at** (set by a database trigger)	Newest-first listing. The service only flips is_published.
6	**products.public_description**	The private description may contain internal notes. Customers only see this text.
7	**products.contact_for_price**	Owner can hide the price and show 'Contact for price'.
8	**products.pta_status** (PTA_APPROVED, NON_PTA, empty)	Pakistani phone buyers ask for it. Empty for accessories.
9	**Separate store phone and WhatsApp numbers**	users.phone is the owner's login. It must never become public.
10	**URL rules in the database**: only http(s) links	Blocks javascript: and data: links that would be unsafe on a public page.
11	**Partial index** on products (shop_id, published_at) where published and active	Fast public catalog.
12	**Backfill**: every existing shop gets a disabled store row with a unique slug	Slug cannot be empty. Existing shops are ready the day Release 1.2 is switched on. The store stays invisible until the owner enables it.

What did NOT change

All Release 1 and 1.1 tables and rules, the cancel-with-reason design, the append-only ledgers, and the no-delete triggers. Products are still deactivated (is_active = false), never deleted. An inactive product never shows on the store, even if is_published is true. A suspended shop's store is hidden as well.

Public versus private data

The public store may only show fields from this allowlist. Anything not listed here must not appear in any public response. Public routes use an explicit field list (Prisma select) and always filter by the shop from the slug.

Store information (from shop_store_settings)

Public	Never public
slug, display_name, about, logo_url, banner_url, whatsapp_number, phone_number, address, city, google_maps_url, opening_hours, facebook_url, instagram_url	shop_id, shops.status, subscription_ends_at, shops.phone and shops.address (private copies), users.* (name, phone, password fields), show_exact_stock itself (only its effect), is_enabled

Products (from products, brands, categories, stock, images)

Public	Never public
id, name, brand name, category name, selling price (or 'contact for price'), images, public_description, model, color, ram, storage, pta_status, condition (NEW / USED) for used phones, availability status, exact quantity only if show_exact_stock is on	default_purchase_price, unit_cost, avg_unit_cost, profit or margin, supplier or supplier payments, IMEI numbers, SKU, barcode, private description, internal notes, shop_id, created_by, is_active, cancel data, customers, any sale or purchase record

Which products appear

- The shop found by slug must exist, have is_enabled = true, and have status ACTIVE (not suspended).
- The product must have **is_published = true AND is_active = true** and belong to that same shop. A product id from another shop returns 404.
- All three failures (no store, store off, shop suspended) return the same 'store not available' answer, so nobody can tell them apart.

How availability is calculated

Product type	Source	Result
QUANTITY (chargers, covers)	stock_levels.quantity_on_hand	0 = OUT_OF_STOCK. 1 up to low_stock_threshold = LOW_STOCK. Above the threshold = IN_STOCK.
SERIALIZED (phones)	Count of inventory_units with status IN_STOCK (one grouped query for the whole page)	Same thresholds as above.

Exact numbers are shown only when the owner turns show_exact_stock on. By default the store shows the status only, so competitors cannot read the shop's inventory.

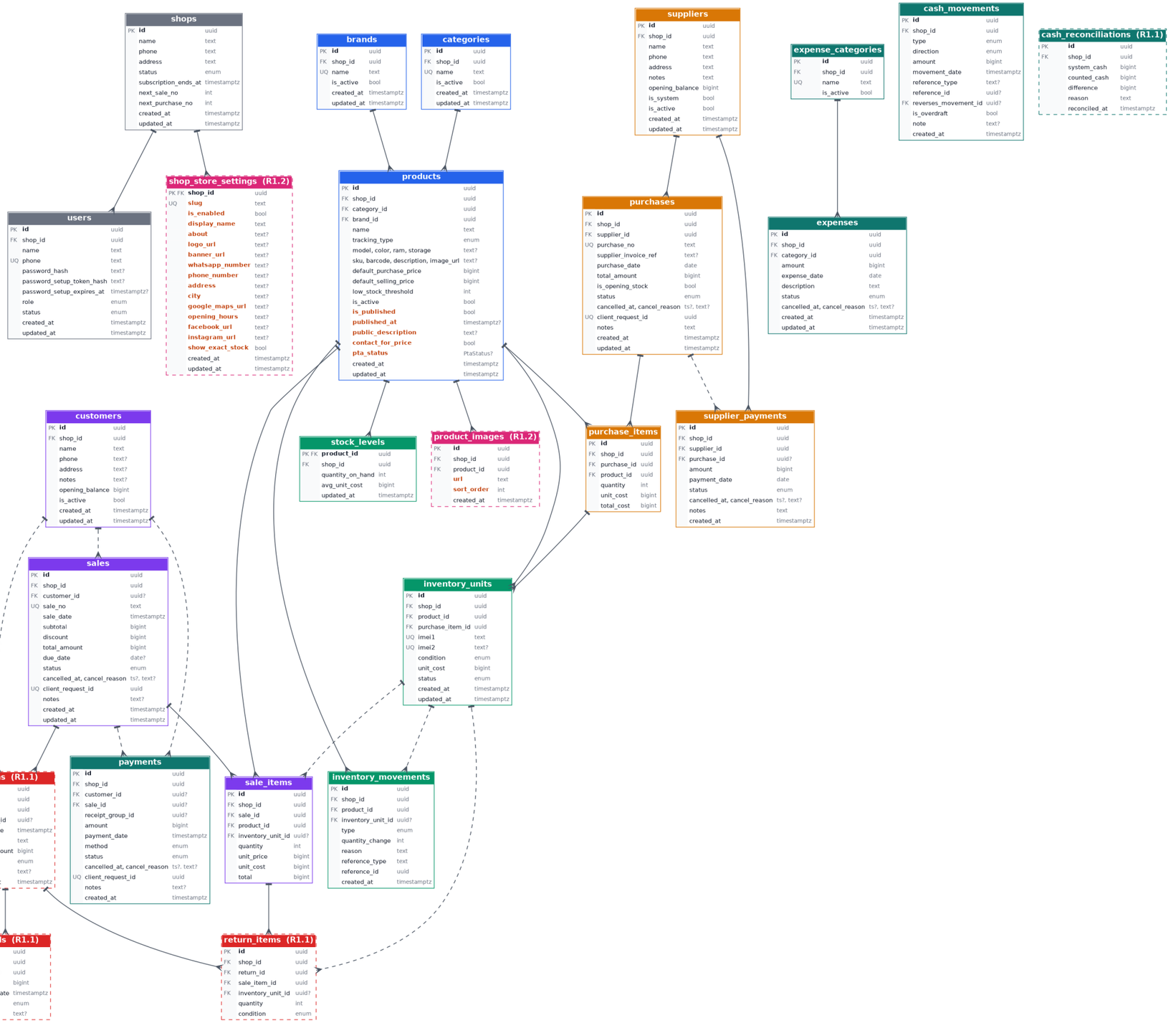
New database rules (Release 1.2)

Rule	How it is enforced	Where
Slug is unique across all shops	UNIQUE (slug)	DB
Slug format	CHECK: lowercase letters, numbers, single hyphens, 3 to 40 characters	DB
Reserved slugs blocked	CHECK against a list (www, app, api, admin, store, stores, static, assets, cdn, mail, support, help, dashboard, login, signup, public, status, dukaan, dukaanpro, dukanpro)	DB
Slug generation without clashes	Service tries the slug, and on a unique-constraint error adds -2, -3 and retries	App + DB
Slug changes	Only an admin may change a slug (shared links break otherwise)	App
WhatsApp number format	CHECK: 10 to 15 digits, country code included, no + or spaces	DB
Only safe links	CHECK: logo, banner, map, Facebook, Instagram and product image URLs must start with http:// or https://	DB
published_at maintained	Trigger: set when is_published becomes true, cleared when false	DB
Store row never deleted	Trigger (reuses forbid_delete)	DB
Max images per product	Service rule (suggested 5)	App
Public routes never leak fields	Explicit select lists, DTOs for responses, and automated tests that check forbidden fields are absent from every public response	App
Fast public catalog	Partial index products_public_catalog_idx (shop_id, published_at DESC) where is_published and is_active	DB

The Release 1 rules in constraints.sql (negative stock, append-only ledgers, no deleting business records, cancel needing a reason, and so on) stay exactly as they were.

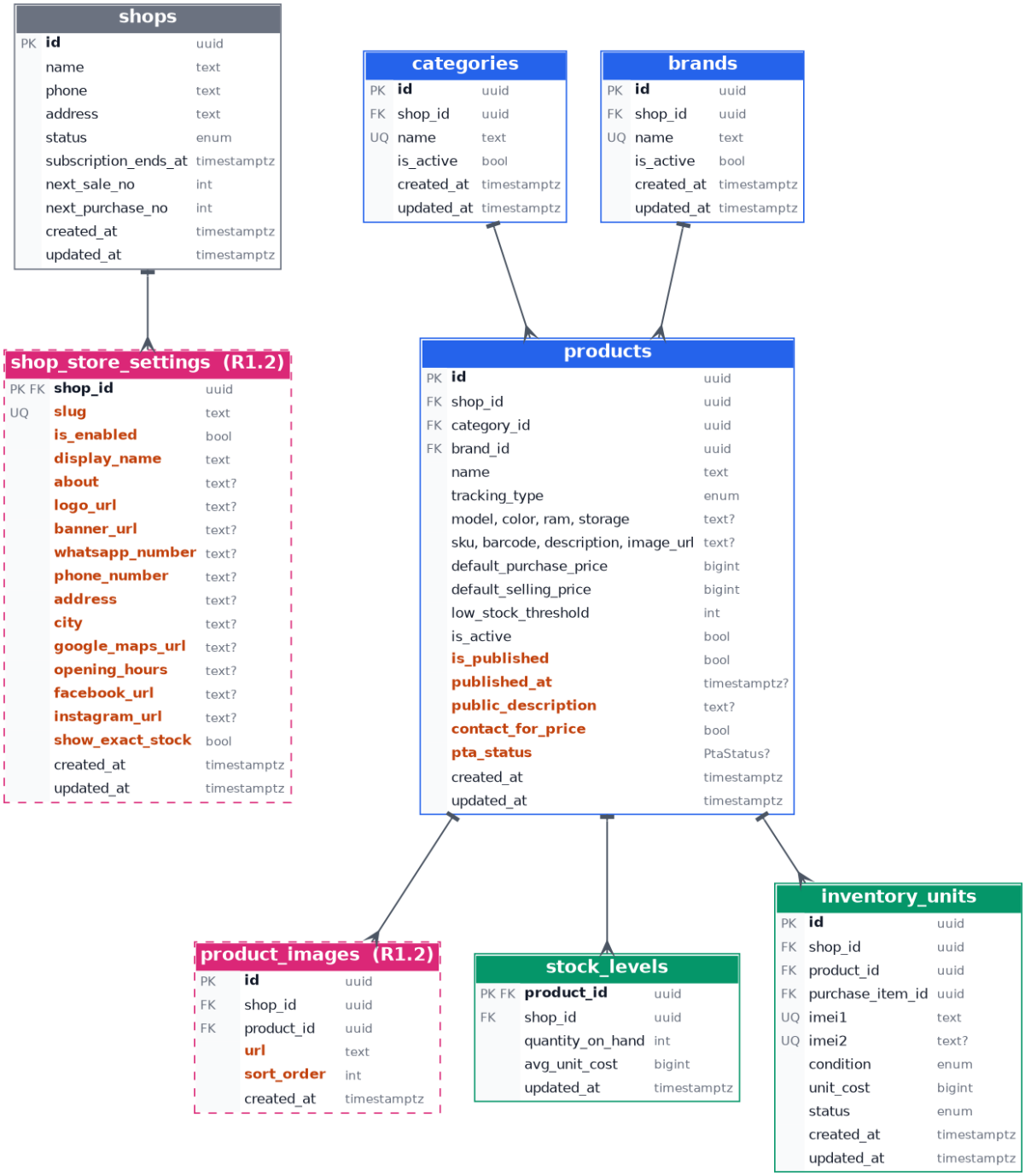
Full ERD v2.1 - 25 shop tables (19 Release 1, 4 Release 1.1, 2 Release 1.2)

Every table except *shops* also has shop_id (FK to shops.id). Orange bold = new in v2.1. Dashed table = built in a later release.

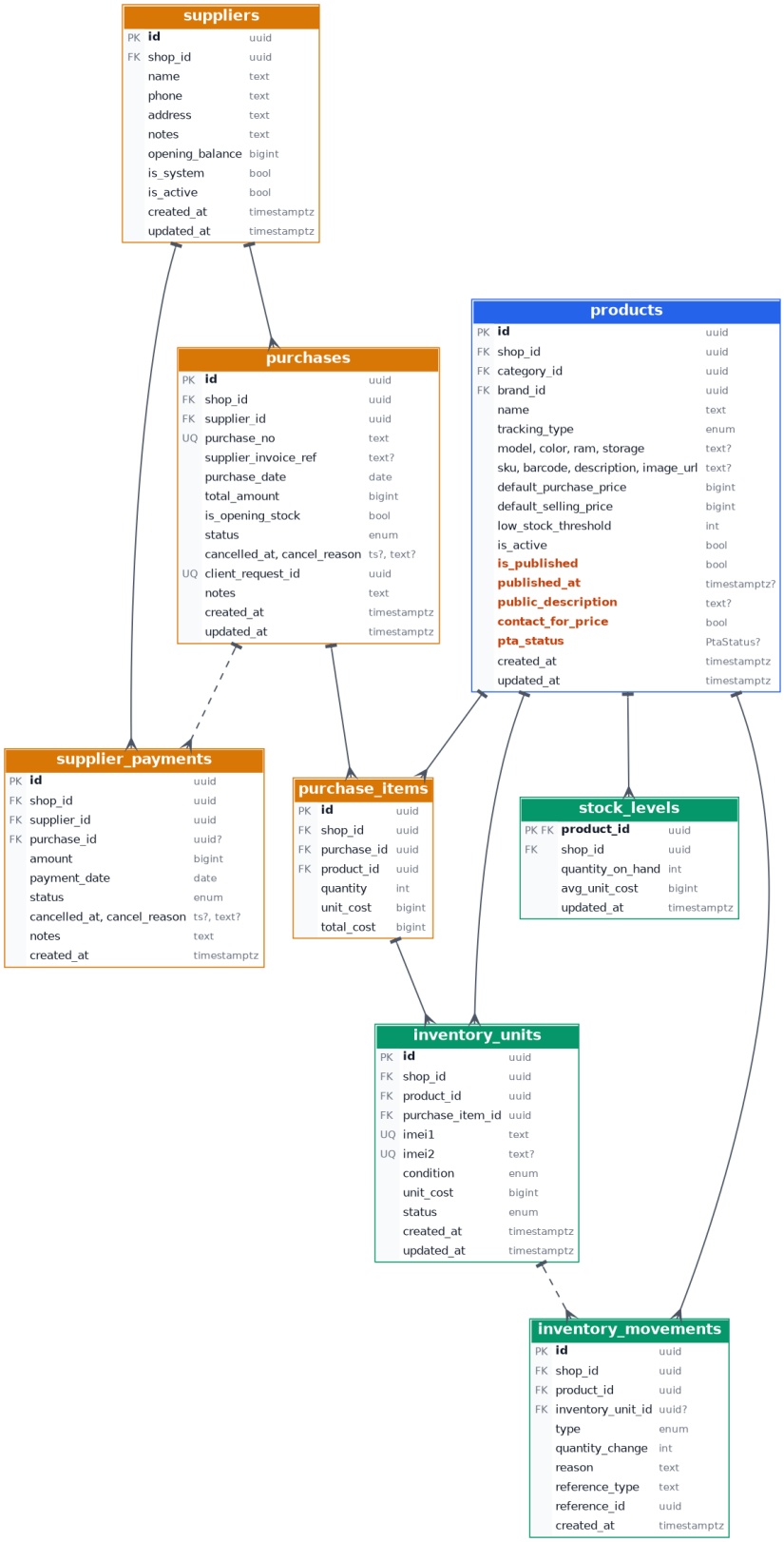


Focused view 1: Online Store (Release 1.2)

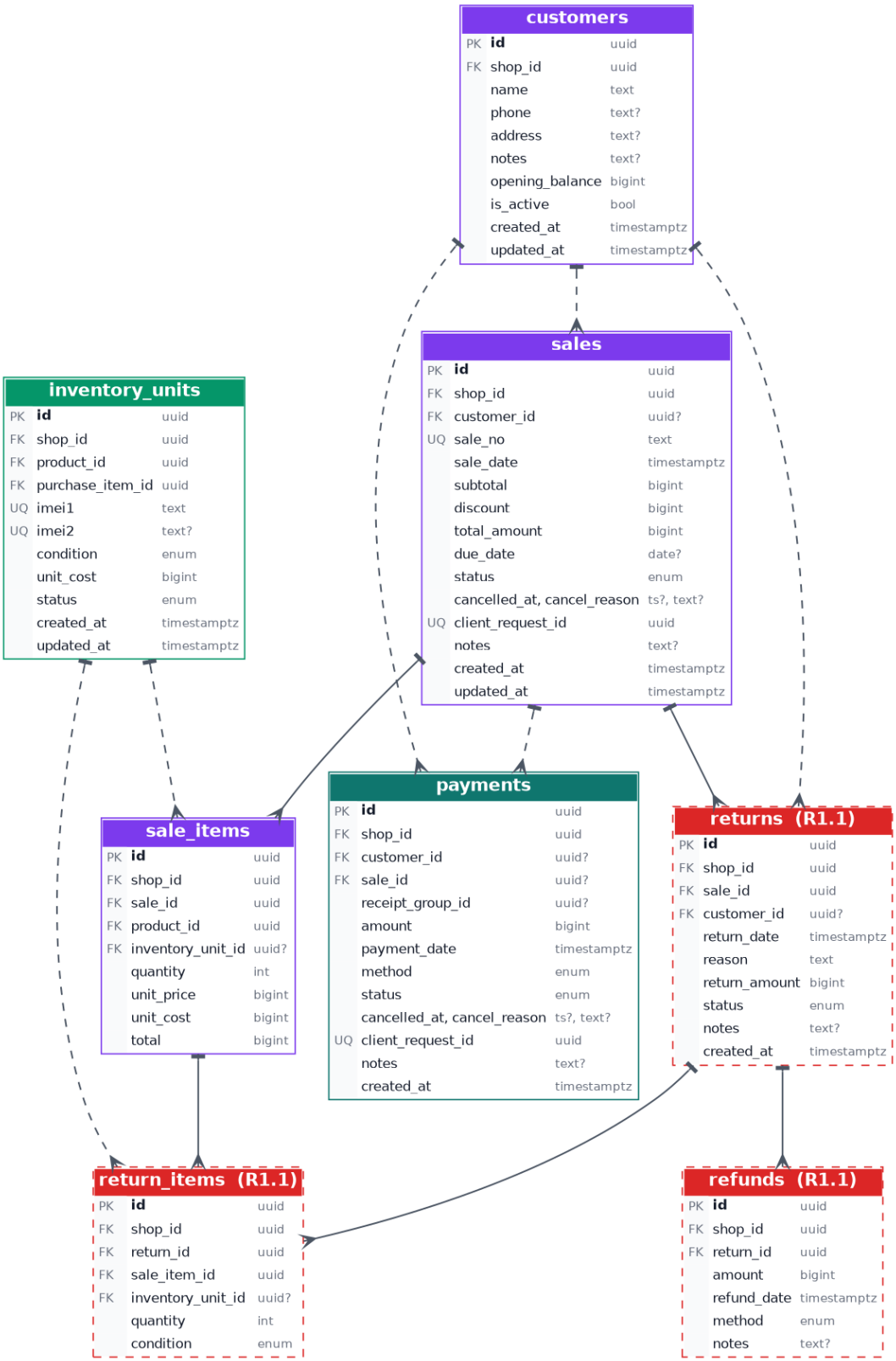
A shop has one **shop_store_settings** row (public profile, slug and on/off switch). Products get publishing fields, and **product_images** holds the photos. Availability is not stored: it is calculated from **stock_levels** (accessories) or the count of IN_STOCK **inventory_units** (phones). Prices, costs, suppliers and IMEIs stay private.



Focused view 2: Purchasing and Inventory (unchanged)



Focused view 3: Sales, Payments and Returns (unchanged)



Example: Samsung A15 goes online

Step	What happens in the database
1	Khalid turns on the store. shop_store_settings: slug khalid-mobile, is_enabled = true, WhatsApp 923001234567.
2	He adds two photos. product_images: 2 rows (sort_order 0 and 1).
3	He writes a public description and presses Publish. products: is_published = true, published_at set by the trigger, contact_for_price = false.
4	A customer opens khalid-mobile.dukaanpro.com. The API finds the shop by slug, checks the store is enabled and the shop is ACTIVE, and returns the product with: name, brand, price 4,500,000 paisa (Rs 45,000), images, public description, availability.
5	3 phones IN_STOCK with threshold 2 = IN_STOCK. After Ali buys one, 2 remain = LOW_STOCK. After the last sale, 0 = OUT_OF_STOCK. Nobody edits the product; availability follows stock automatically.
6	The customer taps WhatsApp. The link opens a chat with a prefilled message and the product link. No order is created in the system.
7	Khalid deactivates the product, or his subscription ends. The product, or the whole store, disappears from the public API immediately.

Table dictionary

Columns tagged V3 are new in v2.1. shop_id on every table is a required foreign key to shops.id unless noted.

shops - Release 1

Column	Type	Key / note
id	uuid	PK
name	text	
phone	text	
address	text	
status	enum	
subscription_ends_at	timestamptz	
next_sale_no	int	
next_purchase_no	int	
created_at	timestamptz	
updated_at	timestamptz	

Rule: Top-level tenant. Status ACTIVE / SUSPENDED is separate from user status. The two counters create automatic numbers (S-0001, P-0001); they are increased inside the same transaction that saves the sale or purchase.

users - Release 1

Column	Type	Key / note
id	uuid	PK
shop_id	uuid	FK to shops
name	text	
phone	text	unique
password_hash	text?	
password_setup_token_hash	text?	
password_setup_expires_at	timestamptz?	
role	enum	
status	enum	
created_at	timestamptz	
updated_at	timestamptz	

Rule: Phone is unique across the platform (login). password_hash stays empty until the owner sets it through the one-time setup link (also used for admin-assisted recovery). MVP: one OWNER per shop.

categories - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
name	text	unique per shop
is_active	bool	
created_at	timestamptz	
updated_at	timestamptz	
Rule: Default categories are copied into each shop. Unique (shop_id, name).		

brands - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
name	text	unique per shop
is_active	bool	
created_at	timestamptz	
updated_at	timestamptz	
Rule: Shop-specific. Unique (shop_id, name).		

products - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
category_id	uuid	FK to categories
brand_id	uuid	FK to brands
name	text	
tracking_type	enum	
model, color, ram, storage	text?	optional
sku, barcode, description, image_url	text?	optional
default_purchase_price	bigint	
default_selling_price	bigint	
low_stock_threshold	int	
is_active	bool	
is_published	bool	**V3**
published_at	timestamptz?	**V3**
public_description	text?	**V3**
contact_for_price	bool	**V3**
pta_status	PtaStatus?	**V3**
created_at	timestamptz	
updated_at	timestamptz	

Rule: tracking_type = QUANTITY or SERIALIZED and cannot change after the first transaction. Default prices are suggestions only. Never hard-deleted once used. Release 1.2: is_published (default false), published_at (set by a trigger), public_description (the only text customers see; description stays private), contact_for_price, pta_status (PTA_APPROVED / NON_PTA / empty).

product_images - RELEASE 1.2		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
product_id	uuid	FK to products
url	text	**V3**
sort_order	int	**V3**
created_at	timestamptz	

Rule: RELEASE 1.2. Photos for the storefront. Lowest sort_order is the cover photo. Max 5 per product (service rule). URL must start with http(s). Images may be deleted (they are not financial records).

shop_store_settings - RELEASE 1.2		
Column	**Type**	**Key / note**
shop_id	uuid	PK ; FK to shops
slug	text	unique ; **V3**
is_enabled	bool	**V3**
display_name	text	**V3**
about	text?	**V3**
logo_url	text?	**V3**
banner_url	text?	**V3**
whatsapp_number	text?	**V3**
phone_number	text?	**V3**
address	text?	**V3**
city	text?	**V3**
google_maps_url	text?	**V3**
opening_hours	text?	**V3**
facebook_url	text?	**V3**
instagram_url	text?	**V3**
show_exact_stock	bool	**V3**
created_at	timestamptz	
updated_at	timestamptz	

Rule: RELEASE 1.2. One row per shop. Every field is public, or controls what is public. slug is globally unique, lowercase letters, numbers and single hyphens, 3 to 40 characters, and cannot be a reserved word (www, app, api, admin ...). Links must start with http(s). WhatsApp number is digits only with country code. The owner's login phone (users.phone) is never used here. A row is never deleted: turn the store off with is_enabled = false.

suppliers - Release 1

Column	Type	Key / note
id	uuid	PK
shop_id	uuid	FK to shops
name	text	
phone	text	
address	text	
notes	text	
opening_balance	bigint	
is_system	bool	
is_active	bool	
created_at	timestamptz	
updated_at	timestamptz	

Rule: is_system = true marks the default supplier 'Walk-in seller' (used phones bought from people). It cannot be deleted or renamed.

purchases - Release 1

Column	Type	Key / note
id	uuid	PK
shop_id	uuid	FK to shops
supplier_id	uuid	FK to suppliers
purchase_no	text	unique per shop
supplier_invoice_ref	text?	
purchase_date	date	
total_amount	bigint	
is_opening_stock	bool	
status	enum	
cancelled_at, cancel_reason	ts?, text?	optional
client_request_id	uuid	unique per shop
notes	text	
created_at	timestamptz	
updated_at	timestamptz	

Rule: purchase_no is automatic (P-0001). supplier_invoice_ref is the supplier's own paper number (optional). Paid amount is NOT stored; it is the sum of active supplier_payments. Opening-stock purchases create no payable and no cash movement. client_request_id stops double saves.

purchase_items - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
purchase_id	uuid	FK to purchases
product_id	uuid	FK to products
quantity	int	
unit_cost	bigint	
total_cost	bigint	

Rule: unit_cost is the real cost at that time and never changes. CHECK quantity > 0, unit_cost >= 0.

supplier_payments - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
supplier_id	uuid	FK to suppliers
purchase_id	uuid?	FK to purchases (optional)
amount	bigint	
payment_date	date	
status	enum	
cancelled_at, cancel_reason	ts?, text?	optional
notes	text	
created_at	timestamptz	

Rule: purchase_id is empty when paying the supplier's opening balance. Total active payments for a purchase cannot exceed its total_amount. A wrong payment is cancelled with a reason (cash reversal), not edited.

inventory_units - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
product_id	uuid	FK to products
purchase_item_id	uuid	FK to purchase_items
imei1	text	unique per shop
imei2	text?	unique per shop
condition	enum	
unit_cost	bigint	
status	enum	
created_at	timestamptz	
updated_at	timestamptz	

Rule: One row per physical phone. condition = NEW or USED. Status: IN_STOCK, SOLD, DAMAGED, LOST, VOIDED (VOIDED = purchase was cancelled). Only IN_STOCK can be sold. Unique (shop_id, imei1).

stock_levels - Release 1		
Column	**Type**	**Key / note**
product_id	uuid	PK ; FK to products
shop_id	uuid	FK to shops
quantity_on_hand	int	
avg_unit_cost	bigint	
updated_at	timestamptz	

Rule: Current quantity for QUANTITY products (chargers, covers). CHECK quantity_on_hand >= 0. Updated in the same DB transaction as every movement. avg_unit_cost is the simple average cost used for profit.

inventory_movements - Release 1

Column	Type	Key / note
id	uuid	PK
shop_id	uuid	FK to shops
product_id	uuid	FK to products
inventory_unit_id	uuid?	FK to inventory_units (optional)
type	enum	
quantity_change	int	
reason	text	
reference_type	text	
reference_id	uuid	
created_at	timestamptz	

Rule: Append-only history. Types: PURCHASE, SALE, SALE_CANCEL, PURCHASE_CANCEL, CUSTOMER_RETURN (R1.1), DAMAGED, LOST, ADJUSTMENT. Reason is required for adjustments, damage and loss.

customers - Release 1

Column	Type	Key / note
id	uuid	PK
shop_id	uuid	FK to shops
name	text	
phone	text?	
address	text?	
notes	text?	
opening_balance	bigint	
is_active	bool	
created_at	timestamptz	
updated_at	timestamptz	

Rule: Walk-in accessory sales use customer_id = NULL. A sale that contains a phone requires a customer (app rule).

sales - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
customer_id	uuid?	FK to customers (optional)
sale_no	text	unique per shop
sale_date	timestamptz	
subtotal	bigint	
discount	bigint	
total_amount	bigint	
due_date	date?	
status	enum	
cancelled_at, cancel_reason	ts?, text?	optional
client_request_id	uuid	unique per shop
notes	text?	
created_at	timestamptz	
updated_at	timestamptz	

Rule: sale_no is automatic (S-0001). Discount is sale-level only. total = subtotal - discount. Paid is NOT stored; it is the sum of active payments. If outstanding > 0 the customer is required. CANCELLED sales are ignored in Udhar, profit and reports.

sale_items - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
sale_id	uuid	FK to sales
product_id	uuid	FK to products
inventory_unit_id	uuid?	FK to inventory_units (optional)
quantity	int	
unit_price	bigint	
unit_cost	bigint	
total	bigint	

Rule: unit_price and unit_cost are saved at sale time. The old item-level discount column was removed (V2): discount is only on the sale. Phone = quantity 1 with inventory_unit_id. Accessory = quantity N, inventory_unit_id NULL. Selling below cost shows a warning but is allowed.

payments - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
customer_id	uuid?	FK to customers (optional)
sale_id	uuid?	FK to sales (optional)
receipt_group_id	uuid?	
amount	bigint	
payment_date	timestamptz	
method	enum	
status	enum	
cancelled_at, cancel_reason	ts?, text?	optional
client_request_id	uuid	unique per shop
notes	text?	
created_at	timestamptz	

Rule: customer_id is empty for a walk-in cash sale. sale_id is empty when paying old opening-balance Udhar. One Udhar payment can be split across several sales (oldest first); the rows share receipt_group_id so they print as one receipt. Payment cannot exceed outstanding.

expense_categories - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
name	text	unique per shop
is_active	bool	

Rule: Default categories (Rent, Electricity, Internet, Salary, Transport, Maintenance, Marketing, Other) copied per shop.

expenses - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
category_id	uuid	FK to expense_categories
amount	bigint	
expense_date	date	
description	text	
status	enum	
cancelled_at, cancel_reason	ts?, text?	optional
created_at	timestamptz	
updated_at	timestamptz	

Rule: Status ACTIVE or CANCELLED. Cancelled with a reason, never deleted. Net profit uses only ACTIVE expenses.

cash_movements - Release 1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
type	enum	
direction	enum	
amount	bigint	
movement_date	timestamptz	
reference_type	text?	
reference_id	uuid?	
reverses_movement_id	uuid?	FK to cash_movements (optional)
is_overdraft	bool	
note	text?	
created_at	timestamptz	

Rule: Append-only cash ledger. Types: OPENING_CASH, CUSTOMER_PAYMENT, SUPPLIER_PAYMENT, EXPENSE, REVERSAL, plus (Release 1.1) REFUND, OWNER_INVESTMENT, OWNER_WITHDRAWAL, RECONCILIATION_ADJUSTMENT. A cancel adds a REVERSAL row pointing to the original. is_overdraft marks a movement saved after the owner confirmed a 'cash will go below zero' warning.

returns - RELEASE 1.1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
sale_id	uuid	FK to sales
customer_id	uuid?	FK to customers (optional)
return_date	timestamptz	
reason	text	
return_amount	bigint	
status	enum	
notes	text?	
created_at	timestamptz	

Rule: RELEASE 1.1. Original sale is never edited. Reason is required. Until 1.1, mistakes are fixed with Cancel sale.

return_items - RELEASE 1.1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
return_id	uuid	FK to returns
sale_item_id	uuid	FK to sale_items
inventory_unit_id	uuid?	FK to inventory_units (optional)
quantity	int	
condition	enum	

Rule: RELEASE 1.1. condition = GOOD or DAMAGED. A serialized unit can be returned only once (unique on inventory_unit_id).

refunds - RELEASE 1.1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
return_id	uuid	FK to returns
amount	bigint	
refund_date	timestamptz	
method	enum	
notes	text?	

Rule: RELEASE 1.1. Money given back to the customer. Creates a cash OUT movement.

cash_reconciliations - RELEASE 1.1		
Column	**Type**	**Key / note**
id	uuid	PK
shop_id	uuid	FK to shops
system_cash	bigint	
counted_cash	bigint	
difference	bigint	
reason	text	
reconciled_at	timestamptz	

Rule: RELEASE 1.1. Owner counts the drawer, the system compares and saves the difference with a reason.